

O problema de *Minimum Linear Arrangement* em Behavior Trees

Uma solução com Simulated Annealing

Victor R. S. de Oliveira

Otimização & Pesquisa Operacional
PPGI — Universidade Federal de Alagoas (UFAL)

30/06/2026

Motivação

O contexto: BTreeFy

Behavior Trees (BTs) são usadas extensivamente em robótica e sistemas embarcados para controle de comportamento.

BTreeFy armazena os nós como um **array flat 1D** de `btf_node` em C:

```
struct btf_node {  
    uint32_t parent;  
    uint32_t child;  
    uint32_t sibling;  
    // ...  
};
```

O problema: a travessia DFS segue ponteiros inteiros (índices no array).

Nós conectados que estão **longe no array** causam:

- Evicção de linha de cache
- Recarga desnecessária (*cache miss*)
- Degradação de desempenho em MCUs

Como ordenar os nós no array para minimizar esses saltos?

Modelo do Problema

A BT é modelada como um grafo não-dirigido ponderado $G = (V, E, W)$:

Vértices:

- $V = \{0, 1, \dots, n - 1\}$ — um vértice por nó

Arestas (campos LCRS não-nulos):

- $(i, \text{node.parent})$
- $(i, \text{node.child})$
- $(i, \text{node.sibling})$
- Duplicatas removidas $\rightarrow |E| = O(n)$

Pesos: $W_{ij} = 1$ (uniforme — linha de base)

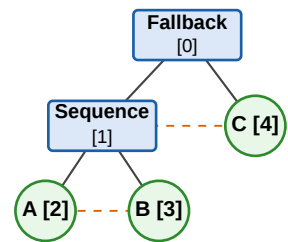
Estrutura LCRS

Left-Child Right-Sibling: cada nó aponta para seu filho mais à esquerda e para o próximo irmão. Minimiza alocação de memória em sistemas embarcados.

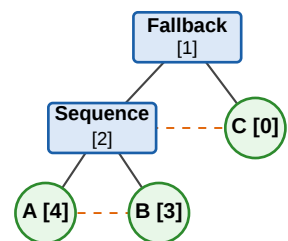
As arestas LCRS criam um grafo esparsos (não-simples) — o grafo não é uma árvore pura devido às arestas de irmão.

Modelo do Problema

Árvore original (DFS pré-ordem) — custo: 12



Layout ótimo (MILP) — custo: 8



Arestas LCRS: $E = \{(0, 1), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3)\}$ — 6 arestas

Layout	Ordem no array	Custo
DFS pré-ordem	[F, S, A, B, C]	12
Ótimo (MILP)	[C, F, S, B, A]	8

Legenda: — parent/child -- sibling

Redução de 33% apenas reordenando o array.

Formulação Matemática

Encontrar uma bijeção $y : V \rightarrow \{0, 1, \dots, n - 1\}$ que minimize:

$$\min_y \sum_{(i,j) \in E} W_{ij} \cdot |y_i - y_j|$$

sujeito a: $y_i \neq y_j \quad \forall i \neq j$

Interpretação:

- y_i = slot (posição) do nó i no array
- $|y_i - y_j|$ = distância entre nós adjacentes na memória
- Minimizar a soma ponderada dessas distâncias

Complexidade

MinLA é **NP-Difícil** para grafos gerais. (Garey & Johnson, 1979)

Grafos BT são árvores esparsas, mas a formulação LCRS (com arestas de irmão) e a extensão para pesos re-introduzem a dificuldade computacional.

Abordagem Exata

Linearização do valor absoluto:

Variável auxiliar $d_{ij} \geq 0$ por aresta:

$$d_{ij} \geq y_i - y_j$$

$$d_{ij} \geq y_j - y_i$$

Objetivo: $\min \sum W_{ij} \cdot d_{ij}$

Slots únicos (Big-M):

Variável binária $z_{ij} \in \{0, 1\}$ por par $i \neq j$:

$$y_i - y_j \geq 1 - n z_{ij}$$

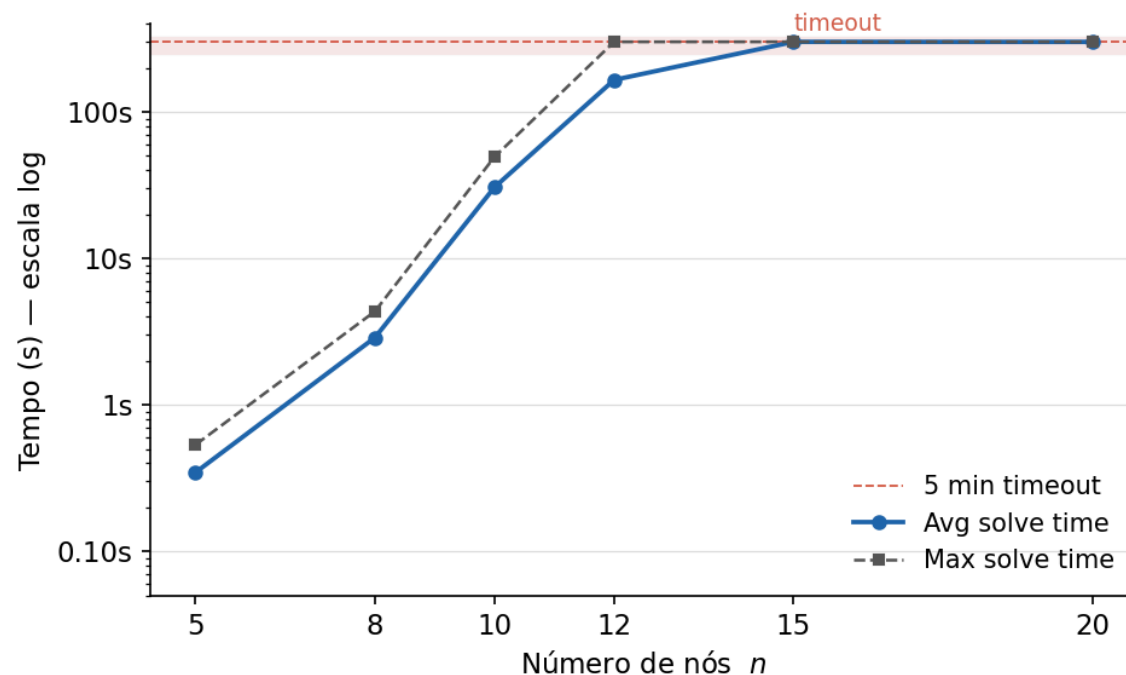
$$y_j - y_i \geq 1 - n(1 - z_{ij})$$

Contagem de variáveis

n	Vars. binárias z_{ij}	Tempo
5	20	< 1 s
10	90	30 s
12	132	173 s
15	210	timeout
20	380	timeout

O crescimento $O(n^2)$ torna o MILP intratável para $n \geq 15$.

O Muro Exponencial



Protocolo:

- $n \in \{5, 8, 10, 12, 15, 20\}$
- 5 instâncias aleatórias por n
- Limite: 300 s (5 min) por instância
- Solver: PuLP / CBC

Observações:

- $n \leq 10 \rightarrow$ sempre ótimo em < 60 s
- $n = 12 \rightarrow$ borda do timeout (83–300 s)
- $n \geq 15 \rightarrow$ **todas as instâncias estouram**

O muro empírico está em $n \approx 12$ –15.

Instâncias reais de BT típicas têm $n = 20$ –200.

Meta-heurística

1. Espaço de estados

Conjunto de todas as **permutações** de n nós.

$|S| = n!$ — intratável por busca exaustiva.

2. Vizinhança

Swap de dois nós aleatórios a, b .

Atualização incremental de custo: $O(\deg(a) + \deg(b))$ — não $O(|E|)$.

3. Aceitação

$\Delta \leq 0$: aceitar sempre.

$\Delta > 0$: aceitar com probabilidade

$$P = e^{-\frac{\Delta}{T}}$$

(critério de Metropolis)

Resfriamento geométrico: $T_{k+1} = \alpha \cdot T_k$, parada em $T < 10^{-4}$. Temperatura inicial T_0 calibrada para 80% de aceitação de pioras.

Meta-heurística

```
Input: grafo  $G$ ,  $\alpha$ ,  $T_{\min}$   
 $y \leftarrow \text{Fiedler\_permutation}(G)$   
 $C \leftarrow \text{MinLA\_cost}(G, y)$   
 $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
 $T \leftarrow \text{initial\_temperature}(G, y)$   
while  $T > T_{\min}$ :  
     $a, b \leftarrow \text{produce\_neighbors\_swap}(G.n)$   
     $\Delta \leftarrow \text{incremental\_cost}(G, y, a, b)$   
    if  $\Delta \leq 0$  or  $\text{rand}() < e^{-\Delta/T}$ :  
        swap  $y_a, y_b$ ;  $C + = \Delta$   
        if  $C < C_{\text{best}}$ :  
             $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
         $T \leftarrow \alpha \cdot T$   
return  $y_{\text{best}}$ 
```

Vetor de Fiedler

1. Montar o Laplaciano ponderado $L = D - A$
2. Computar os autovetores de L (scipy eig)
3. **Vetor de Fiedler** = autovetor do 2º menor autovalor $\lambda_2 > 0$
4. Ordenar nós pelo componente $\rightarrow$ permutação inicial y^0

Pettis & Hansen (1990): ordenação minimiza um *proxy* de transições inter-página — o mesmo que MinLA.

```
Input: grafo  $G$ ,  $\alpha$ ,  $T_{\min}$ 
 $y \leftarrow \text{Fiedler\_permutation}(G)$ 
 $C \leftarrow \text{MinLA\_cost}(G, y)$ 
 $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$ 
 $T \leftarrow \text{initial\_temperature}(G, y)$ 
while  $T > T_{\min}$ :
     $a, b \leftarrow \text{produce\_neighbors\_swap}(G.n)$ 
     $\Delta \leftarrow \text{incremental\_cost}(G, y, a, b)$ 
    if  $\Delta \leq 0$  or  $\text{rand}() < e^{-\Delta/T}$ :
        swap  $y_a, y_b$ ;  $C + = \Delta$ 
        if  $C < C_{\text{best}}$ :
             $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$ 
         $T \leftarrow \alpha \cdot T$ 
return  $y_{\text{best}}$ 
```

No exemplo inicial, permutação inicial com vetor de Fiedler já retorna permutação ótima.

Init	Custo (toy, n=5)
Aleatória	12.0
Fiedler	8.0 — ótimo

```
Input: grafo  $G$ ,  $\alpha$ ,  $T_{\min}$   
 $y \leftarrow \text{Fiedler\_permutation}(G)$   
 $C \leftarrow \text{MinLA\_cost}(G, y)$   
 $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
 $T \leftarrow \text{initial\_temperature}(G, y)$   
while  $T > T_{\min}$ :  
     $a, b \leftarrow \text{produce\_neighbors\_swap}(G.n)$   
     $\Delta \leftarrow \text{incremental\_cost}(G, y, a, b)$   
    if  $\Delta \leq 0$  or  $\text{rand}() < e^{-\Delta/T}$ :  
        swap  $y_a, y_b$ ;  $C \leftarrow C + \Delta$   
        if  $C < C_{\text{best}}$ :  
             $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
         $T \leftarrow \alpha \cdot T$   
return  $y_{\text{best}}$ 
```

Temperatura inicial — `initial_temperature`

Realiza uma *caminhada aleatória* de 100 swaps a partir de y^0 , amostrando apenas os movimentos que **pioram** o custo ($\Delta > 0$). Com a média desses deltas $\overline{\Delta}^+$, aplica a fórmula de Kirkpatrick:

$$T_0 = -\frac{\overline{\Delta}^+}{\ln(p_0)}, \quad p_0 = 0.8$$

Garante **80%** de aceitação de pioras típicas no início — suficiente para escapar de mínimos locais sem tornar a busca puramente aleatória.

```
Input: grafo  $G$ ,  $\alpha$ ,  $T_{\min}$   
 $y \leftarrow \text{Fiedler\_permutation}(G)$   
 $C \leftarrow \text{MinLA\_cost}(G, y)$   
 $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
 $T \leftarrow \text{initial\_temperature}(G, y)$   
while  $T > T_{\min}$ :  
     $a, b \leftarrow \text{produce\_neighbors\_swap}(G.n)$   
     $\Delta \leftarrow \text{incremental\_cost}(G, y, a, b)$   
    if  $\Delta \leq 0$  or  $\text{rand}() < e^{-\Delta/T}$ :  
        swap  $y_a, y_b$ ;  $C + = \Delta$   
        if  $C < C_{\text{best}}$ :  
             $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
         $T \leftarrow \alpha \cdot T$   
return  $y_{\text{best}}$ 
```

Geração de vizinhança — `produce_neighbors_swap`

Para explorar o espaço de busca, a cada iteração, sorteia-se aleatoriamente dois nós distintos da árvore para realizar uma troca (swap) de posições no array y .

O sorteio garante $a \neq b$ rejeitando tentativas onde os nós são iguais, mantendo a simplicidade da amostragem sem viés.

```
Input: grafo  $G$ ,  $\alpha$ ,  $T_{\min}$   
 $y \leftarrow \text{Fiedler\_permutation}(G)$   
 $C \leftarrow \text{MinLA\_cost}(G, y)$   
 $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
 $T \leftarrow \text{initial\_temperature}(G, y)$   
while  $T > T_{\min}$ :  
     $a, b \leftarrow \text{produce\_neighbors\_swap}(G.n)$   
     $\Delta \leftarrow \text{incremental\_cost}(G, y, a, b)$   
    if  $\Delta \leq 0$  or  $\text{rand}() < e^{-\Delta/T}$ :  
        swap  $y_a, y_b$ ;  $C \leftarrow C + \Delta$   
        if  $C < C_{\text{best}}$ :  
             $C_{\text{best}} \leftarrow C; \quad y_{\text{best}} \leftarrow y$   
         $T \leftarrow \alpha \cdot T$   
return  $y_{\text{best}}$ 
```

Custo incremental — `incremental_cost`

Após o swap de a e b , somente arestas incidentes a esses nós alteram sua contribuição ao custo. O restante cancela:

$$\Delta = \sum_{j \in N(a) \cup N(b)} \Delta_j$$

$O(\deg(a) + \deg(b))$ por iteração ($\ll O(|E|)$)

Critério de Metropolis:

- $\Delta \leq 0$: aceitar sempre (melhora)
- $\Delta > 0$: aceitar com $P = e^{-\Delta/T}$

Validação

Metodologia:

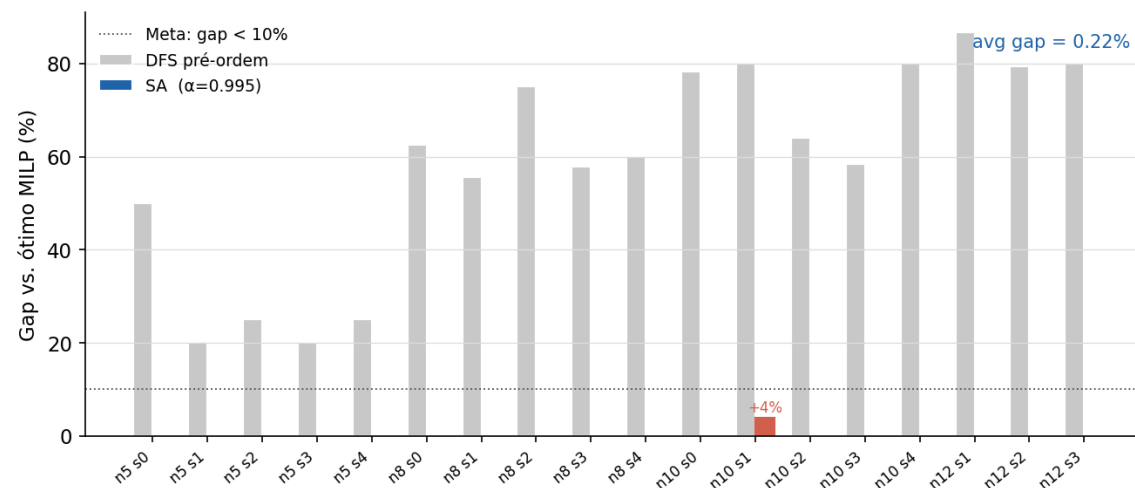
$$\text{Gap}(\%) = \frac{C_{\text{SA}} - C_{\text{MILP}}}{C_{\text{MILP}}} \times 100$$

18 instâncias verificadas ($n \leq 12$,
solve_time < 250 s)

Resultados:

- **17/18** instâncias com gap = 0%
- 1 instância: gap = +4% (n=10, seed=1)
- avg gap = **0,22%**
- max gap = **4,00%**

Muito abaixo da meta de 10% definida no plano do projeto.



Aplicação Real

Pipeline `btf_layout_emitter.py`:

- 1. Parseia modelo XML (BT)
- 2. Constrói BTGraph (LCRS)
- 3. Executa `solve()` com $\alpha = 0.995$
- 4. Remapeia índices parent/child/sibling por y^*
- 5. Emite `btf_nodes_generated.c` drop-in

```
python3 btf_layout_emitter.py \  
-m models/porta_automatica.xml \  
--alpha 0.995 --seed 42
```

Resultados em BTs reais

BT	n	DFS	SA
PortaAutomatica	20	95	71
AssetTracking	6	16	12

Reduções de **25%** e **18%** no custo MinLA.

`./compile-execute.sh --with-tests` →
BUILD & TESTS PASSED ✓

A semântica da árvore é preservada — apenas a ordem do array muda.

Conclusão

Contribuições:

- **P1 — MILP:** formulação exata; ótimo em 0,19 s para $n = 5$; 33% de redução vs. DFS
- **P2 — Scaling:** muro exponencial empírico documentado em $n \approx 12-15$
- **P3 — SA:** $\alpha = 0.995 + \text{init Fiedler}$; avg gap = 0,22% vs. MILP ótimo
- **P4 — Validação:** emissor C funcional; build e testes passando com layout otimizado

Limitações e trabalhos futuros:

- MinLA é um **proxy** de localidade de cache — não minimiza diretamente cache misses
- Pesos W_{ij} uniformes ($= 1$); extensão natural: pesos guiados por profiling
- SA não escalonado para $n \gg 200$ sem calibração adicional de T_0
- Potencial de melhoria: simetria breaking no MILP, heurísticas construtivas alternativas

Referências

- **Garey, M. R. & Johnson, D. S. (1979).** *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman. — Prova de NP-Dificuldade do MinLA.
- **Pettis, K. & Hansen, R. C. (1990).** Profile guided code positioning. *ACM SIGPLAN Notices*, 25(6), 16–27. — Princípio de localidade espacial; base teórica do vetor de Fiedler.
- **Kirkpatrick, S., Gelatt, C. D. & Vecchi, M. P. (1983).** Optimization by Simulated Annealing. *Science*, 220(4598), 671–680. — Base teórica do SA: critério de Metropolis e cooling schedule.
- **Petit, J. (2011).** Experiments on the minimum linear arrangement problem. *Journal of Experimental Algorithmics*, 8. — Benchmark de escalabilidade do MinLA; metodologia de gap de otimalidade.